Cours d'algorithmique et programmation 3

Said Jabbour

UFR des Sciences
Licence Sciences et Technologie
mentions Mathématiques et Informatique
Semestre 3

Année universitaire 2025–2026

Sommaire

1. Complexité
2. Récursivité
3. La dichotomie
4. Algorithmes de Tri
5. Les listes chaînées
6. Une version itérative pour les listes chaînées
7. Une version récursive pour les listes chaînées
8. Les arbres binaires

Section 1

Présentation de l'unité

Modalités de Contrôle de Connaissances : Algorithmique et Programmation 3

- Deux Contrôles Continu : CC1, CC2
- Un Projet (généralement un jeu) : PRJ
- $CC = \frac{2*CC1+2*CC2+TP}{5}$
- Note Algo3 = CC
- Note SAE Algo3 : Projet

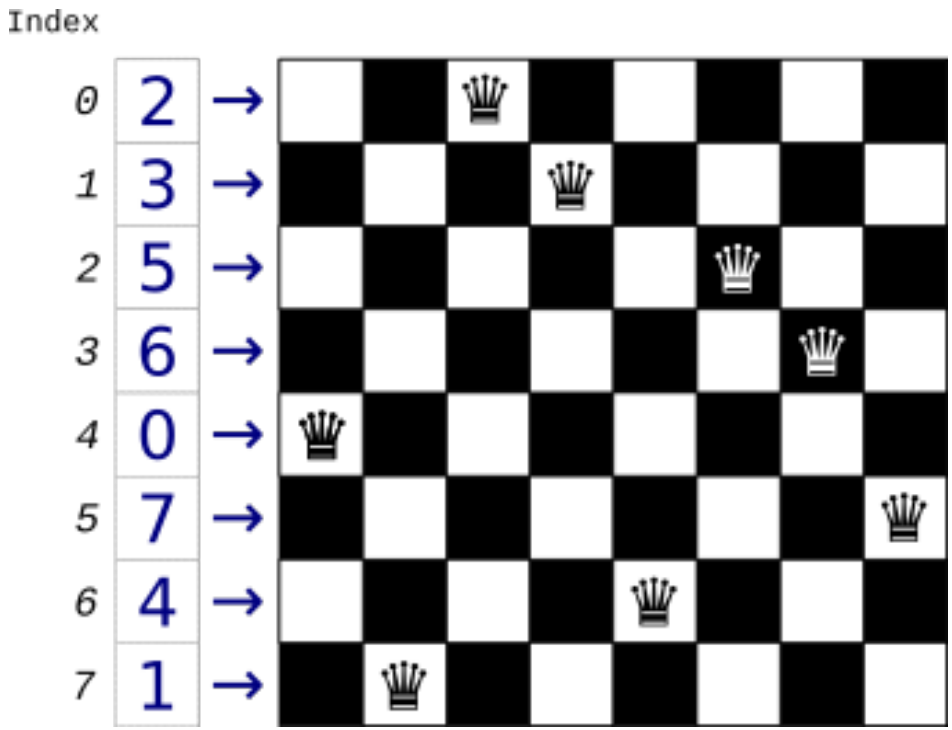
Plusieurs problèmes algorithmiques

- simples
  - Plus grande valeur dans une liste
  - deuxième plus grande valeur dans une liste
  -
- Compliqués
  - **Trier** une liste
  - **résoudre** le problème des *n* reines
  - Parcours du cavalier
  - Fouille de texte
  - Coloriage de graphes, problème de couverture de graphes, etc.

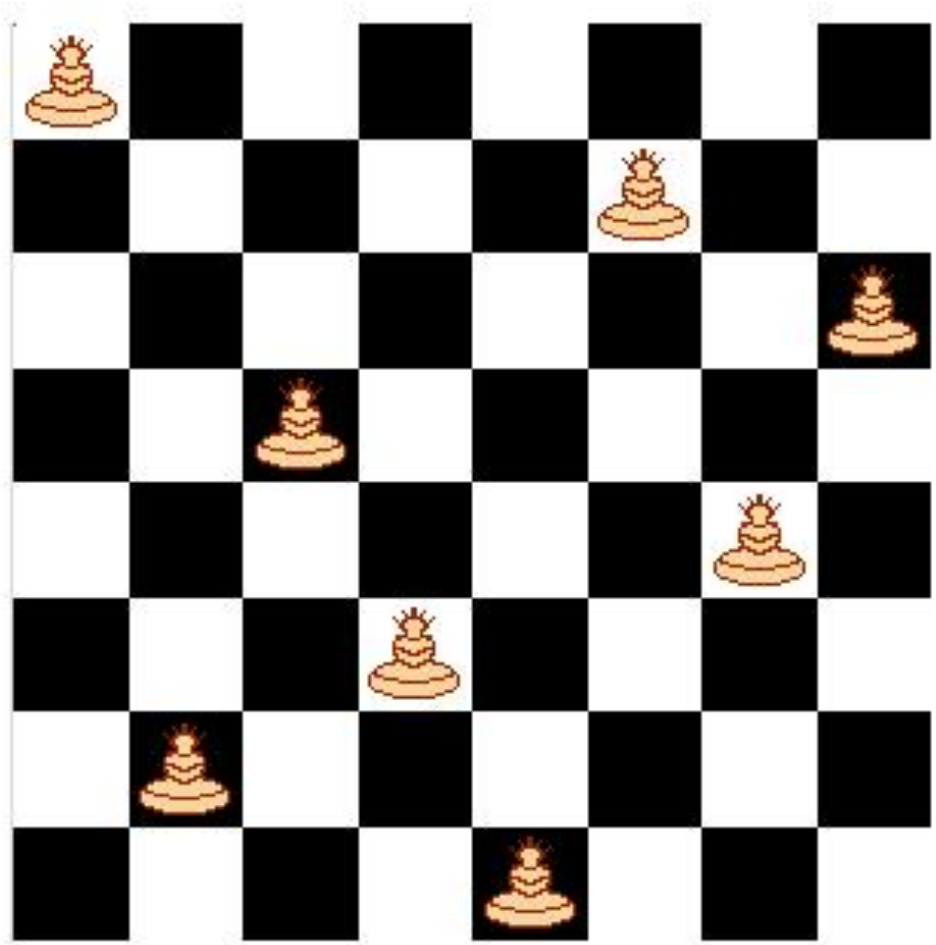
Objectifs

- Solutions algorithmiques pour répondre à ces problèmes
- Utilisation des structures de données adéquates
- Evaluer leurs complexités

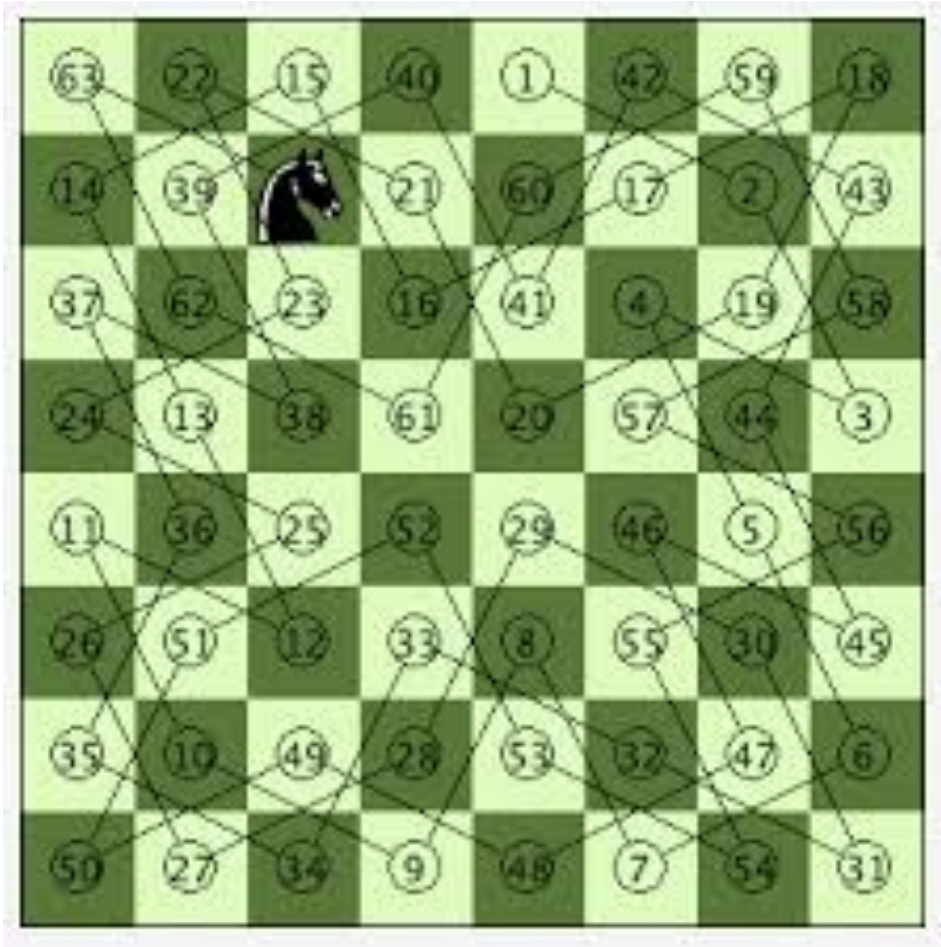
Problème des huit reines



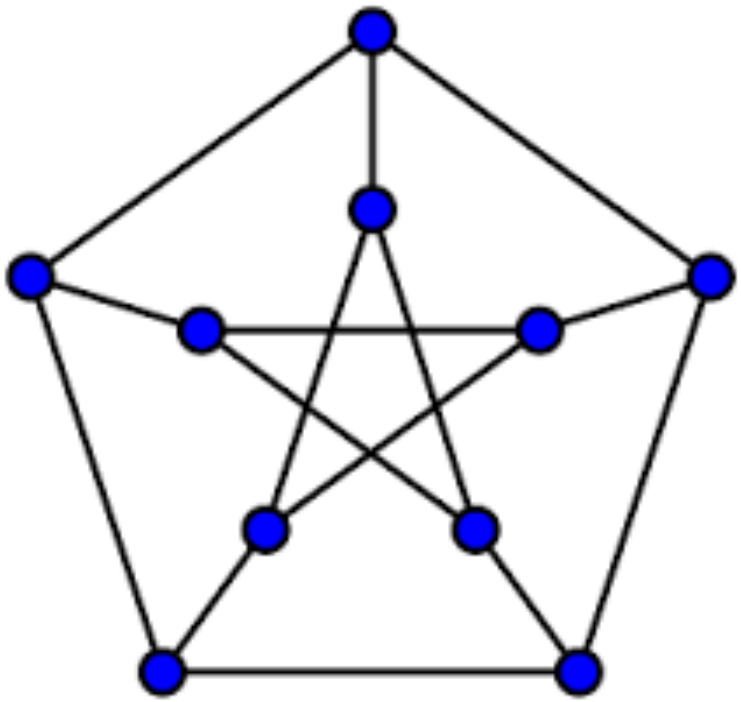
Problème des huit reines



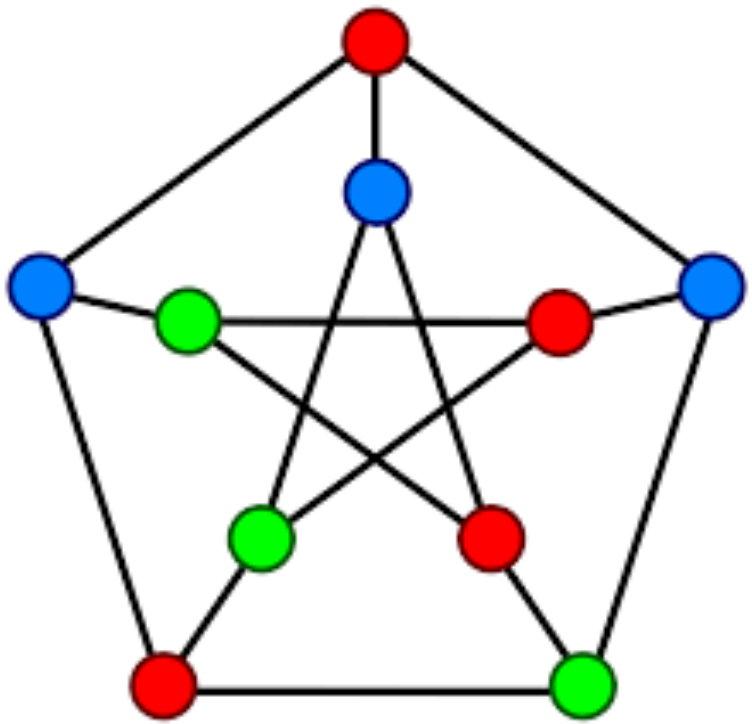
Parcours du cavalier



Colriage de Graphes



Colriage de Graphes

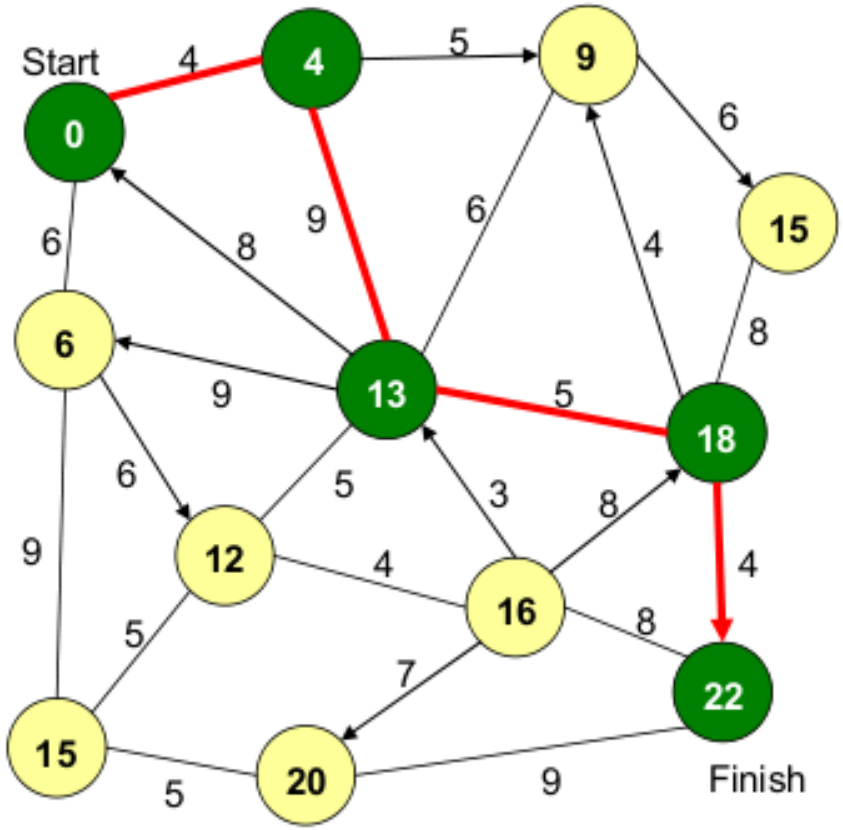


Fouille de motifs ensemblistes

Client 1	oeufs, farine, Levure, sucre
Client 2	oeufs, farine, fromage, yaourt
Client 3	oeufs, farine, course, boisson
Client 4	pattes, yaourt, sauce tomates
Client 5	pattes sauce tomates
Client 6	boisson, cacahuètes, chips

sous-ensembles fréquents (au moins deux fois
{*oeufs*}, {*pattes*, *saucetomates*}, etc.

Plus court Chemin



Sudoku Puzzle

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Résoudre un ensemble d'inéquations linéaires

$$x_1 + x_2 \geq 1$$

$$x_1 - x_2 \geq 0$$

$$-x_1 + x_2 \geq 0$$

$$-x_1 - x_2 \geq -1$$

$x_1 \in \{0, 1\}$, $x_2 \in \{0, 1\}$

Section 2

Complexité

Complexité

Temps de calcul

Ordres de grandeur Comparaisons de complexités

Lequel de ces deux algorithmes est le meilleur ?

```
def min_max_v1(liste):
  mini,maxi = liste[0],liste[0]
  for i in range(1,len(liste)):
    if maxi < liste[i]:
      maxi = liste[i]
    elif liste[i] > mini:
        mini = liste[i]
  return mini,maxi
```

```
def min_max_v2(liste):
  mini,maxi =  liste[0],liste[0]
  for i in range(1,len(liste)):
    if liste[i] < mini:
      mini = liste[i]
  for i in range(1,len(liste)):
    if maxi < liste[i]:
      maxi = liste[i]
  return mini,maxi
```

Temps de calcul

Complexité en temps

$C(A, D)$ = temps d'exécution de l'algorithme A appliqué aux données D.

Eléments de calcul de la complexité en temps

- C(opération élémentaire) = constant
- C(si F alors I sinon J) $C(F) + max(C(I), C(J))$
- C(pourtout i de e_1 à e_2 faire E_i) $\leq C(e_1) + C(e_2) + \sum C(E_i)$
- Temps de calcul de procédures récursives : solution d'équations de récurrence

Temps de calcul (2)

$C(A, D)$ dépend en général de D

- **Complexité au pire**

$$C_{MAX}(A, n) = max\{C(A, D)\ ;\ D\ de\ taille\ n\}$$

- **Complexité au mieux**

$$C_{MIN}(A, n) = min\{C(A, D); D\ de\ taille\ n\}$$

- **Complexité en moyenne**

$$C_{MOY}(A, n) = \sum_{D\ de\ taille\ n} p(D) \times C(A, D)$$

où $p(D)$ est la probabilité d'avoir la donnée D

$$C_{MIN}(A, n) \leq C_{MOY}(A, n) \leq C_{MAX}(A, n)$$

Ordre de grandeur

	$log_2(n)$	$n^{1/2}$	$nlog_2 n$	n^2	n^3	2^n
n=5	2.32	2.24	11.6	25	125	32
n=10	3.32	3.16	33.2	100	1000	1024
n=100	6.64	10	664	10000	10^6	10^{30}
n=1000	9.97	31.62	9970	10^6	10^9	10^{300}
n=10000	13.29	100	132900	10^8	10^{12}	10^{3000}

Ordre de grandeur (2)

Coût $C(n)$	Evolution quand la taille est 10 fois plus grande : $C(n \times 10)$
$log_2(n)$	$C(n) + 3,32$
$n^{1/2}$	$C(n) \times 3,16$
n	$C(n) \times 10$
$nlog_2(n)$	$C(n) \times (10 + e)$
n^2	$C(n) \times 100$
n^3	$C(n) \times 1000$
2^n	$C(n)^{10}$

Comparaisons de complexités : Ordres de grandeur asymptotique

- $C(n) = O(f(n))$

  $\exists k > 0$ et $N \in \mathbb{N}$ tels que $\forall n \geq N$ on a $C(n) \leq k\ f(n)$.

  On dit alors que $C(n)$ est "grand O" de $f(n)$.

- $C(n) = \Omega(f(n))$

  $\exists k > 0$ et $N \in \mathbb{N}$, tels que $\forall n \geq N$, $C(n) \geq kf(n)$

  On dit alors que $C(n)$ est "grand oméga" de $f(n)$.

- $C(n) = \Theta(f(n))$

  si $C(n) = O(f(n))$ et $C(n) = \Omega(f(n))$

  On dit alors que $C(n)$ est "grand thêta" de $f(n)$.

Comparaisons de complexités

Exemples

- $n = O(n^2)$ En effet si $n \geq 1$ on a que $n \leq n^2$ et donc la condition est vérifiée avec $N = 1$ et $k = 1$.
- $log_2(n) = O(n^\alpha)$ pour tout $\alpha > 0$
- $n^\alpha = O(a^n)$ pour tout $\alpha > 0$ et $a > 1$

Popriété

- Si $C(n) = O(f(n))$ alors $f(n) = \Omega(C(n))$

  Preuve : $\forall n \geq N$, on a que $C(n) \leq k \times f(n)$ avec $k > 0$.
  Par conséquent, à partir de N on a que $f(n) \geq 1/k\ C(n)$.

Comparaisons de Complexités

Exemples

$$2n^2 + n = \Theta(n^2)$$

$\forall n \geq 1, 2n^2 + n \leq 2n^2 + n^2 = 3n^2.$
Conséquence : $2n^2 + n = O(n^2)$

$\forall n > 0, 2n^2 + n \geq 2n^2 \rightarrow 2n^2 + n = \Omega(n^2)$

Comparaisons de Complexités

Exemples

$$2n^2 + n = \Theta(n^2)$$

$\forall n \geq 1, 2n^2 + n \leq 2n^2 + n^2 = 3n^2.$
Conséquence : $2n^2 + n = O(n^2)$

$\forall n > 0, 2n^2 + n \geq 2n^2 \rightarrow 2n^2 + n = \Omega(n^2)$

À vous

Quel est la complexité de l'algorithme suivant ?

```
def algo1(liste):
  res = liste[0]
  for i in range(1,len(liste)):
    if res < liste[i]:
      res = liste[i]
  return res
```

Quel est la complexité de l'algorithme suivant ?

```
def algo2(n):
  somme = 0
  for i in range(n-1):
    for j in range(i+1,n):
      somme += i+j
  return somme
```

À vous

Quel est la complexité de l'algorithme suivant ?

```
def algo3(n):
  while n>1:
    print(n)
    n = n // 2
```

Quel est la complexité de l'algorithme suivant ?

```
def alg4(n):
  m = 1
  while m < n:
    print(n)
    m = m * 2
```

Section 3

Récursivité

Le principe de récursivité

Récursivité

Tout objet est dit récursif s'il se définit à partir de lui-même.

Ainsi, une fonction est dite récursive si elle comporte, dans son corps, au moins un appel à elle-même

De même, une structure est récursive si un de ses attributs en est une autre instance

Correspondance mathématique

Principe de récurrence

Exemple

définition des entiers (Peano)

- 0 est un entier
- Si n est un entier, alors n+1 est un entier

Exemples de fonctions récursives

Calcul de la somme des entiers de 1 à n.

- On calcule la somme jusqu'à n-1
- Puis on ajoute n

Idem avec le produit (fonction factorielle)

Un peu de vocabulaire

Pour une fonction récursive, on parlera :

- De récursivité **terminale** si aucune instruction n'est exécutée après l'appel de la fonction à elle-même ou encore
- Une fonction récursive est dite `terminale` si aucun traitement n'est effectué à la remontée d'un appel récursif (sauf le retour d'une valeur).
- De récursivité **non terminale** dans l'autre cas

Terminale / Non Terminale

- Une fonction récursive terminale **est en théorie plus efficace** (mais souvent moins facile à écrire) que son équivalent non terminale : il n'y a qu'une phase de descente et pas de phase de remontée.

- En récursivité terminale, **les appels récursifs n'ont pas besoin d'êtres empilés dans la pile d'exécution** car l'appel suivant remplace simplement l'appel précédent dans le contexte d'exécution.

- Certains langages utilisent cette propriété pour exécuter les récursions terminales aussi efficacement que les itérations.

Récursivité : Exemple

Terminal

```
def f(n):
    if n==0 :
      print ("Hello")
    else :
      f(n-1)
```

Non terminale

```
def f(n) :
    if n>0:
       f(n-1)
     else :
       print ("Hello")
```

Récursivité directe

Lorsque f s'appelle elle-même, on parle de `récursivité directe`

Lorsque f appelle g qui appelle f, il s'agit aussi de récursivité

- On l'appelle alors `indirecte`

Exemples de structures récursives

Liste récursive

- Le premier élément
- Et le reste de la liste (qui est aussi une liste)

Une expression arithmétique est :

- Soit une valeur
- Soit une expression, un opérateur et une autre expression

Implémentation

Comment programmer une fonction récursive ?

Quels sont les pièges à éviter ?

Programmer une fonction récursive

Il suffit de la faire s'appeler elle-même

```
def f(n):
    return f(n-1)
```

La fonction f est récursive : elle s'appelle elle-même

Mais f n'a-t-elle pas un problème ?

Une obligation : s'arrêter

La fonction f telle qu'elle est écrite ne s'arrête pas :

```
1 Appel f(2)
2 Appel f(1)
3 Appel f(0)
4 Appel f(-1)
5 Appel f(-2)
6 etc.
```

Comment y parvenir

Première étape : **la condition terminale**

- Obligatoirement au début de toute fonction récursive
- Une condition : le cas particulier
- Pour ce cas, pas d'autre appel à la fonction : la chaîne d'appels s'arrête

Exemple

```
def f(n):
  if n==0 :
    print ("Hello")
  else :
    f(n-1)
```

Ici quand n vaut 0, on s'arrête

Problème : arrive-t-on à n = 0 ?

Terminaison de la fonction

Il faut que la fonction s'arrête

La condition terminale ne sert à rien si elle ne devient jamais vraie

Exemple avec la fonction précédente :

- f(-2) provoque une pile d'appels infinie
- Probablement d'autres tests à faire (si $n < 0$, envoyer une exception par exemple)

Une bonne solution

```
def f(n) :
  if n < 0 :
    sys.exit(0)
  if n==0 :
    print("Hello")
  else :
    f(n-1)
```

Pourquoi ça marche ?

Si n est négatif : on s'arrête sur une exception

Si n est nul : c'est le cas d'arrêt ("Hello")

Si n est positif : on appelle f avec la valeur n-1

- Chaine d'appels avec des valeurs entières strictement décroissantes de 1 en 1
- On arrive forcément à 0
- On affiche "Hello"
- On remonte la pile des appels (sans rien faire, ici la récursivité est terminale)

Conclusion

Il faut regarder cas par cas, et à la main

Même si aucune méthode n'est générale, le principe de récurrence aide souvent

En résumé

Une fonction récursive doit comporter :

- Un cas d'arrêt dans lequel aucun autre appel n'est effectué
- Un cas général dans lequel un ou plusieurs autres appels sont effectués

La chaîne d'appel doit conduire au critère d'arrêt

- Optionnellement, des cas impossibles ou incorrects à traiter par des exceptions

Quelques exemples

Récursification facile ;

récursivité obligatoire ?

Les boucles for

Toute boucle for peut se transformer en une fonction récursive.

Principe

- Pour faire des choses pour un indice allant de 1 à n
  - On les fait de 1 à n-1 (même traitement avec une donnée différente)
  - Puis on les fait pour l'indice n (cas particulier)

Traduction

```
def f(n) :
  for i in range(n) :
    traiter(i)
```

```
def f(n) :
  if n==0 :
    traiter(0);
  else :
     f(n-1)
     traiter(n)
```

Traduction

```
def tsp(graph,visited,pos,n,count,cost,min_cost):
    # If all vertices are visited, return to the starting v
    if count == n and graph[pos][0]:
        min_cost[0]=min(min_cost[0],cost+graph[pos][0])
        return

    for i in range(n):
        # If the vertex "i" is not visited and there is an
        if not visited[i] and graph[pos][i]:
            visited[i] = True  # Mark the vertex as visited
            tsp(graph,visited,i,n,count+1,cost+graph[pos][i
            visited[i] = False  # Backtrack (unmark the ver
```

Exemple : fonction factorielle

Appel de fact(5) récursif

Version Itérative

```
def fact(n) :
  res = 1
  for i in range(2, n+1) :
    res = res*i
  return res
```

Version Récursive

```
def fact(n) :
  if n == 0 :
    return 1
  else :
    return n*fact(n-1)
```

Appel de fact(5) récursif

Phase de descente récursive

```
1 Appel     fact(5)
2    Appel  fact(4)
3        Appel    fact(3)
4            Appel    fact(2)
5                Appel    fact(1)
6                    Appel    fact(0)
```

Condition terminale

- Retour de la valeur 1

Suite

Phase de remontée (après l'appel à fact(n-1) on multiplie par *n* : la récursivité n'est pas terminale)

```
1  Retour de la valeur 1
2        Retour de la valeur 2
3              Retour de la valeur 6
4                    Retour de la valeur 24
5                          Retour de la valeur 120
```

Quelques conséquences

La plupart des traitement sur les tableaux peuvent se mettre sous forme récursive :

- Tris (sélection, insertion)
- Recherche séquentielle (attention : pas dichotomique)
- Inversion
- Problème des huit reines
- Etc...

Une constatation

L'écriture sous forme récursive est toujours plus simple que l'écriture sous forme itérative

Une question

Même fonction est-elle plus efficace sous forme récursive ou sous forme itérative ? (Ou, sous une autre forme, y a-t-il un choix optimal généralisable ?)

La réponse est non. La réponse à la question inverse est non. Il n'y a pas de généralité

En revanche

La plupart des traitements itératifs simples sont facilement traduisibles sous forme récursive (exemple du for)

L'inverse est faux

Il arrive même qu'un problème ait une solution récursive triviale alors qu'il est très difficile d'en trouver une solution itérative

La fonction d'ackermann

$Ack(m,n) =$

$n+1$	si m=0
$Ack(m-1,1)$	sinon et si n=0
$Ack(m-1,Ack(m,n-1))$	autrement

Remarque : on finit bien car Ack(m,n) est strictement décroissant sur les appels (à l'exception de $Ack(1,0)$ qui finit trivialement)

Exemple

- $u_0 = 1$
- $u_n = 2u_{n-1} + 3$

Version récursive (non terminale)

```
def terme(n):
  if n==0:
    return 1
  return 2*terme(n-1)+3
```

Version terminale

```
def terme(n,acc=1):
  if n==0:
    return acc
  return 2*terme(n-1, 2*acc+3)
```

Version itérative

```
def terme_iter(n):
  val=1
  for i in range (1,n+1):
    val = 2*val+3
  return val
```

Fonctions récursives - À vous !

Exercice

Donner une version itérative, récursive non terminale et récursive terminale de la suite de fibonacci définit comme suit :

- $fibo(0) = 1$
- $fibo(1) = 1$
- $fibo(n) = fibo(n-1) + fibo(n-2), n \geq 2$

Version récursive

```
def fibo(n):
  if n<=1:
    return 1
  return fibo(n-1)+fibo(n-2)
```

Version itérative

```
def fibo(n):
  i=0
  p,q=0,1
  while i < n:
    p,q=q,p+q
    i = i+1
  return q
```

Version terminale

```
def fibo(n,p=0,q=1):
  if n==0:
    return q
  else:
    return fibo(n-1,q,p+q)
```

Écrire une fonction récursive qui compte le nombre d'occurrences d'un caractère dans une chaîne de caractères

```
def nb_occurrences(car,chaine):
  if len(chaine) == 0:
    return 0
  if car == chaine[0]:
    return 1+nb_occurrences(car,chaine[1:])
  return nb_occurrences(car,chaine[1:])
```

```
def nb_occurrences(car,chaine,i=0):
  if i == len(chaine):
    return 0
  if car == chaine[i]:
    return 1+nb_occurrences(car,chaine,i+1)
  return nb_occurrences(car,chaine,i+1)
```

Section 4

La dichotomie

Chercher une valeur dans une liste

Comment faire ?

- parcourir toute la liste
- si la liste est triée, effectuer une recherche dichotomique

Idée : Prendre la valeur au milieu de la liste et comparer avec la valeur cherchée pour savoir

C'est la même idée que pour jouer au nombre mystérieux.

À vous !

L'implémentation de la recherche dichotomique

```
1  def recherche(l,val) :
2      '''liste(int), int -> int
3      retourne l'indice de val si val est presente dans l.
4      retourne -1 sinon.
5      '''
6      deb = 0
7      fin = len(l)
8      while deb < fin :
9          milieu = (deb+fin)//2
10         if l[milieu] == val :
11             return milieu
12         elif l[milieu] < val :
13             deb = milieu+1
14         else :
15             fin = milieu
16     return -1
```

L'implémentation de la recherche dichotomique (2)

et en récursif ? À vous !

```
1     def recherche_rec(l,val,deb,fin) :
2     '''liste(int), int, int, int -> int
3     retourne l'indice de val si val est presente dans l
4     entre l'indice deb inclus et fin exclu.
5     retourne -1 sinon.
6     '''
7     if deb < fin :
8         milieu = (deb+fin)//2
9         if l[milieu] == val :
10            return milieu
11        elif l[milieu] < val :
12            return recherche_rec(l, val, milieu+1, fin)
13        else :
14            return recherche_rec(l, val, deb, milieu)
15    else :
16        return -1
```

avec un appel initial : `recherche_rec(l,val,0,len(l))`

Quelle efficacité ? Combien d'opérations ?

Pour étudier la complexité d'un algorithme, on va s'attacher

- à compter le nombre d'opérations à effectuer,
- souvent en fonction de la taille de la donnée.

On s'intéressera surtout aux ordres de grandeur.

On étudie essentiellement :

- la complexité dans le pire des cas (c'est le plus facile à faire !)
- la complexité en moyenne

Complexité de la recherche d'une valeur dans une liste

- Dans le cas d'une recherche linéaire, i.e. on parcourt toute la liste élément après élément pour essayer de trouver la valeur : au pire n accès et n comparaisons.

  La complexité de la recherche par parcours simple est en $O(n)$ ou de complexité linéaire.

Nombre d'éléments	Nombre de comparaisons
4	4
10	10
50	50
10 000	10 000
100 000	100 000

Complexité de la recherche d'une valeur dans une liste

- Dans le cas d'une recherche dichotomique : à chaque étape, on divise l'espace de recherche par deux ⇒ le nombre d'étapes (accès, comparaison) est l'entier immédiatement supérieur à $log_2(n)$.

  La complexité de la recherche dichotomique est en $O(log(n))$ ou de complexité logarithmique.

Nombre d'éléments	Nombre de comparaisons
4	2
10	4
50	6
10 000	14
100 000	17

Section 5

Algorithmes de Tri

Tri : Objectif

- **Au départ :**
  - Eléments dans un ordre quelconque

5	2	12	3	5	...

- **À la fin :**
  - Les **mêmes** éléments dans l'ordre croissant ou décroissant

2	3	5	9	12	...

Idée du tri

Réaliser des permutations (échanges) d'éléments pour passer de la situation de départ à la situation d'arrivée.

Objectif secondaire

Le moins de permutations possible.

Un premier tri : le tri par sélection

Idée :

- on cherche le plus petit élément de la liste
- on l'insère en début de la liste en permutant sa valeur avec l'élément en tête
- et on recommence

À vous :

- la recherche du plus petit élément
- le tri par sélection

Le tri par sélection - implémentation

```
1  def tri_selection(l) :
2      '''(inout) liste(int) -> rien
3      algo de tri par selection du plus petit element
4      '''
5      for i in range(len(l)) :
6          # recherche du plus petit element
7          ind_mini = recherche_mini(l,i)
8          # permutation a la position i
9          l[i], l[ind_mini] = l[ind_mini], l[i]
10     # fin for
11 # fin tri_selection
```

Le tri par sélection - implémentation (2)

```
1  def recherche_mini(l,ind) :
2      '''liste(int), int -> int
3      recherche l'indice du plus petit element de l
4      qui se trouve entre l'indice ind et la fin de l.
5      '''
6      assert ind < len(l)
7      mini = ind
8      for i in range(ind,len(l)) :
9          if l[i] < l[mini] :
10             mini = i
11     return mini
```

Complexité du tri par sélection

On note n le nombre d'éléments de la liste.
Pour chaque élément d'indice i de la liste, on va faire :

- parcourir n-i éléments restants de la liste
- faire n-i comparaison pour trouver le plus petit élément
- et au pire faire n-i affectations
- plus deux affectations pour permuter l'élément i et le plus petit élément

Soit n(n+1)/2 comparaisons et au pire n(n+1)/2 + 2n affectations.

On simplifie en gardant l'ordre de grandeur : ici, le degré du polynôme.

On dira que le tri par sélection est en $O(n^2)$,
ou qu'il est de complexité quadratique.

Peut-on améliorer le tri par sélection ?

Oui ! Un peu. . .

Idée :

- en même temps qu'on cherche le plus petit élément, on cherche le plus grand
- et à chaque tour on range le plus petit et le plus grand à leur place

Pas de changement pour l'ordre de grandeur de la complexité : toujours quadratique.

Mais la fonction des comparaisons croît moins vite que dans le tri précédent. À vous !

Le tri par sélection - amélioration (1)

```
1  def recherche_min_max(l,deb,fin) :
2      '''liste(int), int, int -> int, int
3      recherche l'indice du plus petit element
4      et du plus grand element de l
5      qui se trouvent entre les indices deb et fin de l.
6      '''
7      assert deb < fin
8      mini = deb
9      maxi = fin - 1
10     for i in range(deb,fin) :
11         if l[i] < l[mini] :
12             mini = i
13         if l[i] > l[maxi] :
14             maxi = i
15     return mini, maxi
```

Le tri par sélection - amélioration (2)

```
1  def tri_selection_min_max(l) :
2      '''(inout) liste(int) -> rien
3      algo de tri par selection du plus petit element
4      et du plus grand element
5      '''
6      prem = 0
7      dern = len(l)
8      while prem < dern :
9          # recherche du plus petit element
10         ind_mini,ind_maxi = recherche_min_max(l,prem,dern
11         # positionnement du plus petit a sa place
12         l[prem], l[ind_mini] = l[ind_mini], l[prem]
13         prem += 1
14         # positionnement du plus grand a sa place
15         dern -= 1
16         l[dern], l[ind_maxi] = l[ind_maxi], l[dern]
17     # fin for
18 # fin tri_selection
```

Le tri à bulles

Le tri à bulles remonte le plus grand élément vers sa place à chaque tour de liste.

L'idée du tri à bulles est la suivante :

On peut profiter de cette recherche du maximum
pour remonter un peu vers le haut les plus grands éléments.

Implémentation du tri à bulles

```
1 def tri_a_bulles(l) :
2     '''(inout) liste(int) -> rien
3     trie l par l'algorithme du tri a bulles
4     '''
5     for i in range(len(l)) :
6         ## on remonte une bulle tout en haut
7         for ind in range(len(l)-i-1) :
8             if l[ind] > l[ind+1] :
9                 l[ind], l[ind+1] = l[ind+1], l[ind]
```

Quelle est la complexité du tri à bulles ?

Implémentation du tri à bulles - amélioration

On peut s'arrêter dès que plus aucune *bulle* n'est remontée !

```
1  def tri_a_bulles(l) :
2      '''(inout) liste(int) -> rien
3      trie l par l'algorithme du tri a bulles
4      '''
5      bulle = True
6      i = 0
7      while i <len(l) and bulle :
8          bulle = False
9          ## on remonte une bulle tout en haut
10         for ind in range(len(l)-i-1) :
11             if l[ind] > l[ind+1] :
12                 l[ind], l[ind+1] = l[ind+1], l[ind]
13                 # des qu'une permutation a eu lieu
14                 # on le marque dans bulle
15                 bulle = True
16         i += 1
```

Le tri par insertion

L'idée du tri par insertion est la suivante :

- on prend chaque élément l'un après l'autre
- et on l'insère à sa place dans la partie du tableau qui est déjà triée

À vous !

Implémentation du tri par insertion

```
1  def tri_insertion(l) :
2      '''(inout) liste(int) -> rien
3      trie l par l'algorithme du tri par insertion
4      '''
5      for i in range(1,len(l)):
6          ref = l[i]
7          # on insere ref dans la liste l[0:i-1]
8          # en decalant les plus grands elements
9          # jusqu'a la position i
10         j = i-1
11         while j >= 0 and l[j] > ref :
12             l[j+1] = l[j]
13             l[j] = ref
14             j = j-1
```

Le tri fusion

Idée :

- fusionner deux listes listes triées en une seule est facile... Comment faire ? Quelle est la complexité ?
- trier une liste, c'est trier les deux moitiés de la liste, et fusionner le résultat !

À vous !

- écrivez la fonction `fusionner(l,deb,milieu,fin)`
- cette fonction recopie les tronçons à fusionner avant de mettre à jour `l`

La fusion (suite)

```
def triFusion(a) :
  #renvoi  la liste des elements de a tries
  n = len(a)
  if n <= 1:
    return a
  m = n // 2
  return fusion(triFusion(a[0:m]),triFusion(a[m:n]))
```

La fusion (suite)

```
def fusionner(a,b) :
  c = []
  n,m = len(a),len(b)
  i,j = 0,0
  #on continue tant que les deux listes ne sont pas vides
  while i < len(a) and j < len(b):
     if a[i] < b[j] :
       c.append(a[i])
       i = i + 1
     else :
       c.append(b[j])
       j = j + 1
  if i == len(a):
    for k in range(j,len(b)):
      c.append(b[k])
  else:
    for k in range(i,len(a)):
      c.append(a[k])
  return c
```

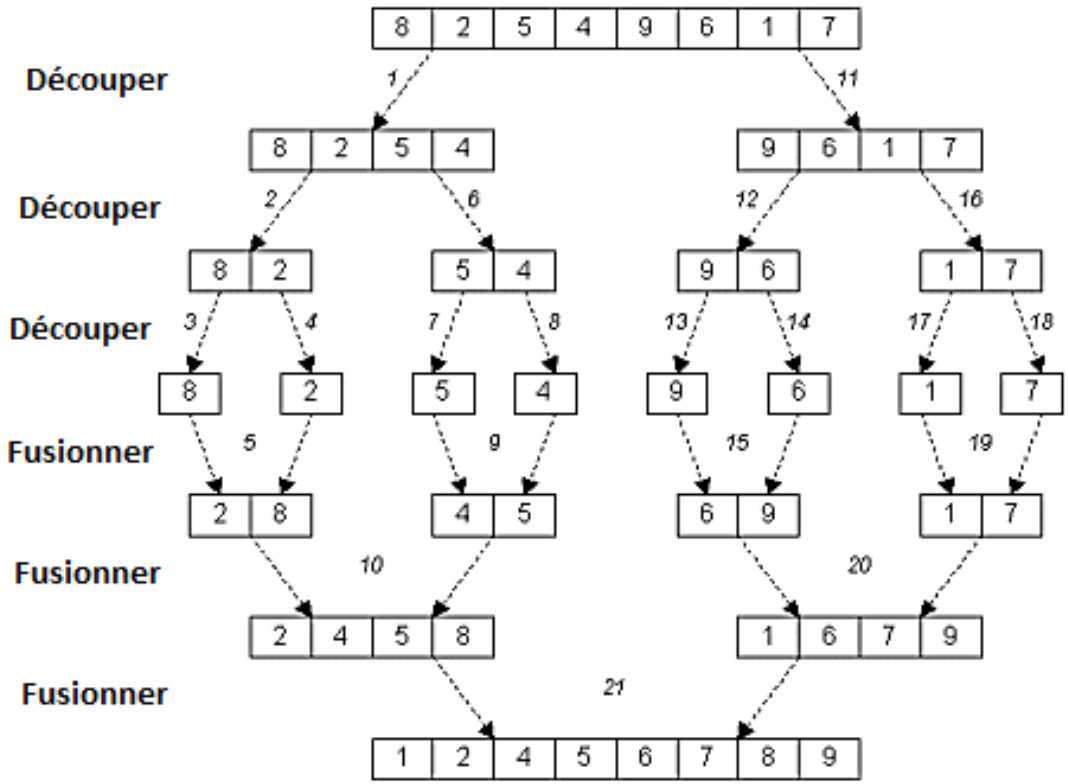
Implémentation de la fusion

```
1  def fusionner(l,deb,milieu,fin):
2      '''(inout) list(int),int,int,int
3      [def:milieu[ et [milieu:fin[ sont tries au depart.
4      l'intervalle [deb:fin[ de l sera trie en sortie.'''
5      part1, part2 = l[deb:milieu], l[milieu:fin]
6      i, j, k = 0, 0, deb
7      while i < len(part1) and j < len(part2) :
8          if part1[i] < part2[j] :
9              l[k] = part1[i]
10             i += 1
11         else :
12             l[k] = part2[j]
13             j += 1
14         k += 1
15     while i < len(part1) :
16         l[k] = part1[i]
17         i, k = i+1, k+1
18     while j < len(part2) :
19         l[k] = part2[j]
20         j, k = j+1, k+1
```

Implémentation du tri fusion

```
1  def tri_fusion(l,deb,fin) :
2      '''(inout) list(int) -> rien
3      trie l par l'algorithme du tri fusion.
4      '''
5      if fin-deb < 2 :
6          return
7      else :
8          milieu = (deb+fin)//2
9          tri_fusion(l,deb,milieu)
10         tri_fusion(l,milieu,fin)
11         fusionner(l,deb,milieu,fin)
```

Illustration



Exemple

Une fois le tableau annexe rempli, il faut le recopier dans le tableau initial

- À faire à chaque étape
- Avec les bons indices
- Attention aux paramètres des appels récursifs

Algorithme efficace ?

Compexité

Si on note $C(n)$ le nombre d'opérations qu'il faut pour trier un tableau de taille n, on a :

$$C(n) = 2 \times C(\frac{n}{2}) + n$$

(n opérations pour la fusion)

- **Si on résout la récurrence, on obtient que** :

Solution

$$C(n) = \Theta(n\ log_2(n))$$

Algorithme efficace (2)

Pour un tri, $n\ log_2(n)$ **est la complexité temporelle optimale**

Pourtant, ce n'est pas le tri le plus utilisé ! !

- Pas la meilleure constante ($k \times n \times log_2(n)$)
- **Surtout** : on a besoin d'un deuxième tableau aussi grand que le premier (problème de complexité spatiale)

Le tri rapide ou Quicksort

Principe

Algorithme

Exemple

Un peu de culture générale

Proposé par Charles Antony Richard Hoare : 1962

C’est un tri dichotomique

Le principe du tri rapide

- On va **séparer le tableau en deux parties**, puis les trier
- **Différence avec le tri fusion**
  - la séparation ne sera pas arbitraire
- **Conséquence :** On n'aura plus besoin de fusionner

Principe de la séparation

1. On choisit une valeur du tableau, qui servira de **pivot**

2. On réorganise le tableau de façon à ce que :

   Toutes les valeurs inférieures au pivot soient placées avant lui dans le tableau et toutes les valeurs supérieures après

Implantation

```
def triRapide(liste,debut,fin):
    if debut < fin:
        # partitionnement de la liste
        pivot = partition(liste, debut, fin)
        # trier les deux semi-listes
        triRapide(liste,debut,pivot-1)
        triRapide(liste,pivot+1,fin)
    return liste
```

```
def partition(liste,debut,fin):
  pivot = liste[debut]
  i = debut+1
  j = fin
  fait = False
  while not fait:
    while i <= j and liste[i] <= pivot:
      i += 1
    while liste[j] >= pivot and i <= j:
      j-=1
    if j < i:
      fait= True
    else:
      #permuter
      liste[i],liste[j]=liste[j],liste[i]
  #permute debut avec liste[fin]
  liste[debut],liste[j]=liste[j],liste[debut]
  return j
```

Quelques remarques

Ici on considère que le premier élément est le pivot

→ Si ce n'est pas lui, un premier échange suffit

On avance dans le tableau en remettant systématiquement les valeurs inférieures au pivot dans les premières cases

Le dernier échange met le pivot à sa place

Autres Méthodes

Deux itérateurs :

le premier part de début+1

le second part de la fin (le pivot étant encore au début)

1. Tant qu'on est plus petit que le pivot en partant du début, on avance
2. Tant qu'on est plus grand en partant de la fin, on recule
3. Si les deux positions ne sont pas confondues, on échange
4. Puis on recommence jusqu'à ce que les positions soient confondues

Ensuite

- **Une fois le tableau partitionné** :
- on trie les deux "moitiés" (encore de la récursivité)
- autour du pivot (exclus, il est déjà à sa place)

- **Une fois qu'elles sont triées, pas besoin de fusionner**
  - Les éléments du sous-tableau de gauche sont forcément inférieurs à ceux du sous-tableau de droite

- **Cas d'arrêt ?** encore une fois évident :
  - moins de 2 cases

Pourquoi ça marche

On prend un pivot

Les plus petits devant, les plus grands derrière

Et on trie les deux parties autour du pivot

Algorithme efficace

Fortement dépendant du choix du pivot

Choix optimal ? clé médiane

Pire des choix : le plus petit ou le plus grand

- On se retrouve grosso modo au tri par sélection

Mais un choix du pivot optimal passe par un autre algorithme

En pratique

On tape au hasard

Pire des cas : on prend toujours le plus grand ou le plus petit

Dans ce cas, **on est en** $\Theta(n^2)$

Complexité moyenne

$$\Theta(n\ log_2(n))$$

Avec une constante inférieure à celle du tri fusion

Ne nécessite pas de tableau annexe ! ! !

Efficace quand ?

- **Pour les structures de grande taille**

- Pour les petits tableaux, on lui préfère un tri simple, comme la sélection ou l'insertion

Optimisation

en dessous d'une certaine taille limite, on ne fait plus l'appel récursif mais un tri par insertion

autre implémentation de la partition

```
1  def partitionner(l,deb,fin) :
2      '''(inout) list(int), int, int -> int
3      partitionne la liste et retourne l'indice du pivot.
4      '''
5      assert deb < fin-1
6      pivot = l[fin-1]
7      borne_inf = deb
8      borne_sup = fin-1
9      while borne_inf < borne_sup :
10         if l[borne_inf] < pivot :
11             borne_inf += 1
12         else :
13             l[borne_sup] = l[borne_inf]
14             l[borne_inf] = l[borne_sup-1]
15             borne_sup -= 1
16     l[borne_inf] = pivot
17     return borne_inf
```